

Chapter 6 — Practice Log

Problems, Mistakes & Fixes

A record of the hands-on drills worked through after Chapter 6 — the problems attempted, the mistakes made along the way, and exactly how each one was diagnosed and fixed. The mistakes are the valuable part: each one exposes a concept that is easy to misread from the book alone.

1 Problem 1 — Frequency Report

Task: From a file of repeated fruit names, produce a report of each unique fruit with its count, sorted most-frequent first.

Mistake: `uniq` counted 1 of everything

```
$ cat fruits.txt | uniq -c
 1 banana
 1 apple
 1 cherry
 1 apple
 ...
```

Every line came back with a count of 1, even though `apple` appeared four times.

Why It Happened

`uniq` only collapses duplicate lines that are **adjacent**. It compares each line to the one directly before it — it never scans the whole file. Since no two identical fruits were sitting next to each other, every line looked “different from the one above” and got counted as 1.

The Fix

Run `sort first` so identical lines become adjacent, then `uniq -c` can group and count them:

```
$ sort fruits.txt | uniq -c
```

Mistake: trusting a coincidental order

After `sort | uniq -c`, the output happened to already be in most-frequent order — but only because, for that data, alphabetical order and frequency order lined up. Using a deliberately “adversarial” data set (where `date` was most frequent but sorts last alphabetically) exposed that the counts alone were not in frequency order.

The Fix — Complete Pipeline

Add a second sort, numeric and reversed, then redirect to a file:

```
$ sort fruits.txt | uniq -c | sort -nr > report.txt
```

Read it as: group identical lines → count each group → re-sort by count descending → save.

Mistake: `-r` without `-n`

Sorting with `-r` alone reverses, but compares the counts as *text*, character by character — so `10` would sort as smaller than `9`. It looked fine only because the data had single-digit counts.

Lesson — Always Pair `-nr` for Numbers

`-n` reads the values as numbers; `-r` puts the biggest first. Keep them together (`sort -nr`) as a habit, or double-digit counts will silently sort wrong.

2 Problem 2 — Top Errors from a Log

Task: From a log mixing `ERROR`, `INFO`, and `WARN` lines, produce the two most frequent `ERROR` messages with counts.

Mistake: `grep ERROR*` and wrong pipeline order

```
$ sort server.log | grep ERROR* | uniq -c
```

This produced the right-looking output, but for two shaky reasons.

Why It Was Fragile

(1) **`ERROR*` is a regex, not a wildcard.** In `grep`, the pattern is a regular expression. `*` means “zero or more of the preceding character,” so `ERROR*` means “ERRO followed by any number of R’s.” It matched only by luck. The correct plain-substring match is simply `grep ERROR`.

(2) **Pipeline order was backwards.** Sorting *before* filtering works but reads awkwardly. The natural order is filter first, then group, then count.

The Fix

```
$ grep ERROR server.log | sort | uniq -c | sort -nr | head -n 2 > top_errors.txt
```

Filter to `ERROR` lines → sort → count → order by frequency → keep the top 2 → save.

Glob `*` vs. Regex `*` — The Trap That Bites Everyone

Globbering is done by the *shell* on filenames *before* the command runs; `*` matches any run of characters.

Regex is interpreted by the *program* (`grep`, `sed`, ...) on text inside lines; `*` means “zero or more of the previous character” and needs something in front of it to modify.

Same symbol, completely different meaning.

3 Problem 3 — `tee` and `wc`

Task: Build a login-count summary that is written to a file *and* shown on screen at once, then separately count total login lines.

Result: correct on the first try

```
$ sort logins.txt | uniq -c | sort -nr | tee summary.txt
$ wc -l logins.txt
10 logins.txt
```

What Went Right

`tee` correctly tapped the stream — displaying the result live while simultaneously saving it — and a follow-up `cat summary.txt` confirmed the file held identical content. The count-and-sort pipeline was applied cleanly with no coincidence-riding.

Refinement: filename label vs. bare number

`wc -l logins.txt` prints `10 logins.txt` — the count *and* the filename. To get just the number, feed the file via `stdin` instead of naming it:

```
$ wc -l < logins.txt      # prints: 10
$ cat logins.txt | wc -l  # prints: 10
```

Why the Filename Appears or Disappears

Given a *filename argument*, `wc` opens the file itself and echoes its name. Given a nameless `stdin` stream (via `<` or a pipe), it has no name to report, so it prints only the number.

4 Problem 4 — `tail` and `Append`

Task: Save the 3 most recent events from a timestamped log, then append the total line count below them without erasing them.

Result: solved cleanly, better than the model answer

```
$ tail -n 3 activity.log > recent.txt
$ wc -l < activity.log >> recent.txt
$ cat recent.txt
09:15 dave login
09:30 alice login
09:45 carol logout
8
```

What Went Right

Three correct judgments in a row: `tail -n 3` grabbed the most recent events; `» appended` the count instead of overwriting (using `>` here would have wiped the three lines and left only 8); and `wc -l < activity.log` was chosen over `wc -l activity.log`, producing a clean bare 8 with no filename — applying the `stdin`-stream principle unprompted.

The Make-or-Break Distinction

`>` overwrites (truncates the file first). `»` appends (adds to the end). When adding to an existing file, using the wrong one silently destroys the previous contents.

5 Running Themes

Patterns Worth Carrying Forward

1. **The workhorse pipeline:** `sort | uniq -c | sort -nr` — group, count, rank. Used everywhere: top IPs in a log, most common errors, biggest offenders in any list.
2. `uniq` **needs sorted input** because it only sees adjacent lines.
3. **Pair** `-nr` whenever sorting numbers, or double digits sort wrong.
4. **Glob** `*` $\neq$ **regex** `*` — shell filenames vs. grep text patterns.
5. `>` **destroys**, `»` **adds** — respect the difference.
6. **Filename argument vs. `stdin` stream** determines whether a tool can see a name; commands treat piped/redirected input as an anonymous byte stream.
7. **Beware coincidental correctness** — test pipelines on data where the “easy” order and the “correct” order disagree.